

Big Data Analysis

(Análise de Dados de Grandes Dimensões)

Apache Spark Modules



Apache Spark Ecosystem

◎ Data Science and Machine Learning



◎ SQL analytics and BI

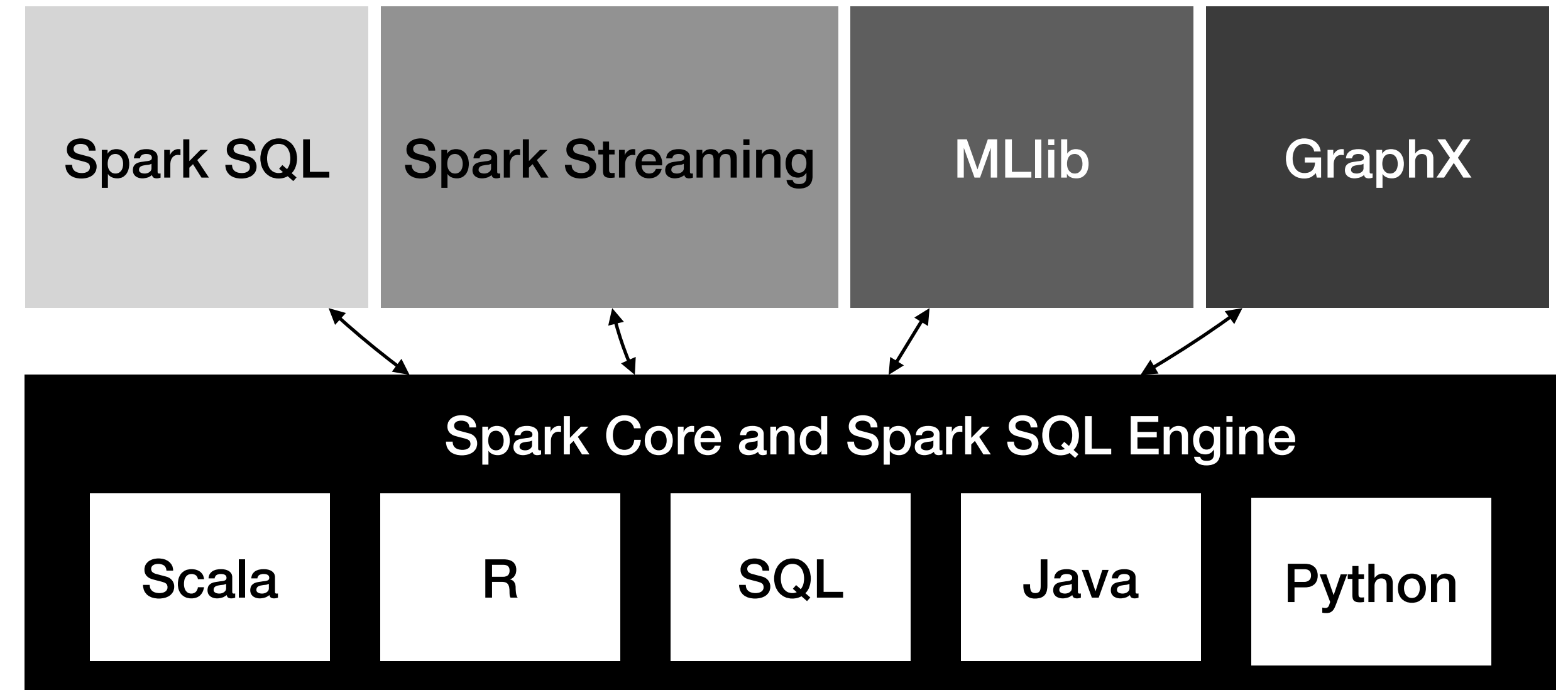


◎ Storage and Infrastructure



Apache Spark Modules

- Spark SQL and Dataframes + Datasets
- Spark Streaming - Structured Streaming
- Machine Learning - **MLlib**
- Graph Processing - **GraphX**



Spark Streaming (Structured Streaming)

- Structured Streaming enables users to build **continuous applications**.
 - A continuous application is an end-to-end application that reacts to data in real-time by combining various tools: streaming jobs, batch jobs, joins between streaming and offline data, and interactive ad-hoc queries.
- It treats a **stream** of data **as a table** to which **data is continuously appended**.

Spark Streaming (Structured Streaming)

◎ **Input sources:**

- Files in distributed file systems like HDFS or S3.
- Kafka.
- Socket source (for testing).
- Rate source (for testing).
- Rate Per Micro-Batch source (for testing).

◎ **Sinks:**

- File - stores output to a directory.
- Kafka - stores the output to 1 or more topics.
- Foreach - Runs arbitrary computation on the records in the output.
- Console (for debugging) - Prints the output to the console/stdout.
- Memory (for debugging) - stores the output in memory as an in-memory table.

◎ **Output Modes** (are dependent on the queries and the sinks):

- Append (only add new records to the output sink).
- Update (update changed records in place).
- Complete (rewrite the full output).

◎ **Triggers:**

- By default, Structured Streaming will look for new input records as soon as it has finished processing the last group of input data.
- It can also work based on processing time, such as looking for new data at a fixed interval.

Spark Streaming (Structured Streaming)

```
from pyspark.sql import SparkSession
from pyspark.sql.functions import explode
from pyspark.sql.functions import split

spark = SparkSession \
    .builder \
    .appName("StructuredNetworkWordCount") \
    .getOrCreate()
```

```
# Create DataFrame representing the stream of input lines from
connection to localhost:9999
```

```
lines = spark \
    .readStream \
    .format("socket") \
    .option("host", "localhost") \
    .option("port", 9999) \
    .load()
```

```
# Split the lines into words
```

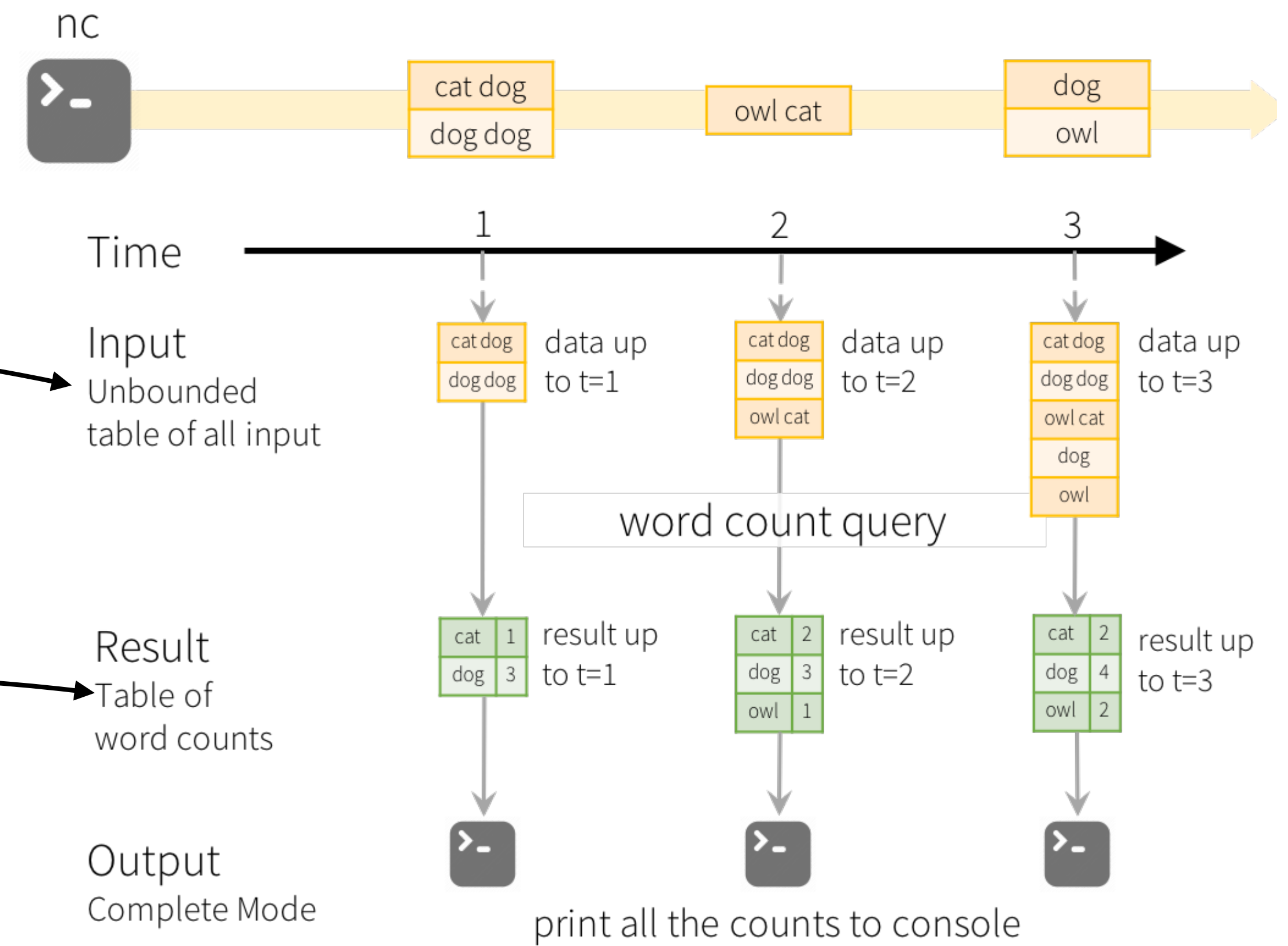
```
words = lines.select(
    explode(
        split(lines.value, " ")
    ).alias("word")
)
```

```
# Generate running word count
```

```
wordCounts = words.groupBy("word").count()
```

```
query = wordCounts \
    .writeStream \
    .outputMode("complete") \
    .format("console") \
    .start()
```

```
query.awaitTermination()
```

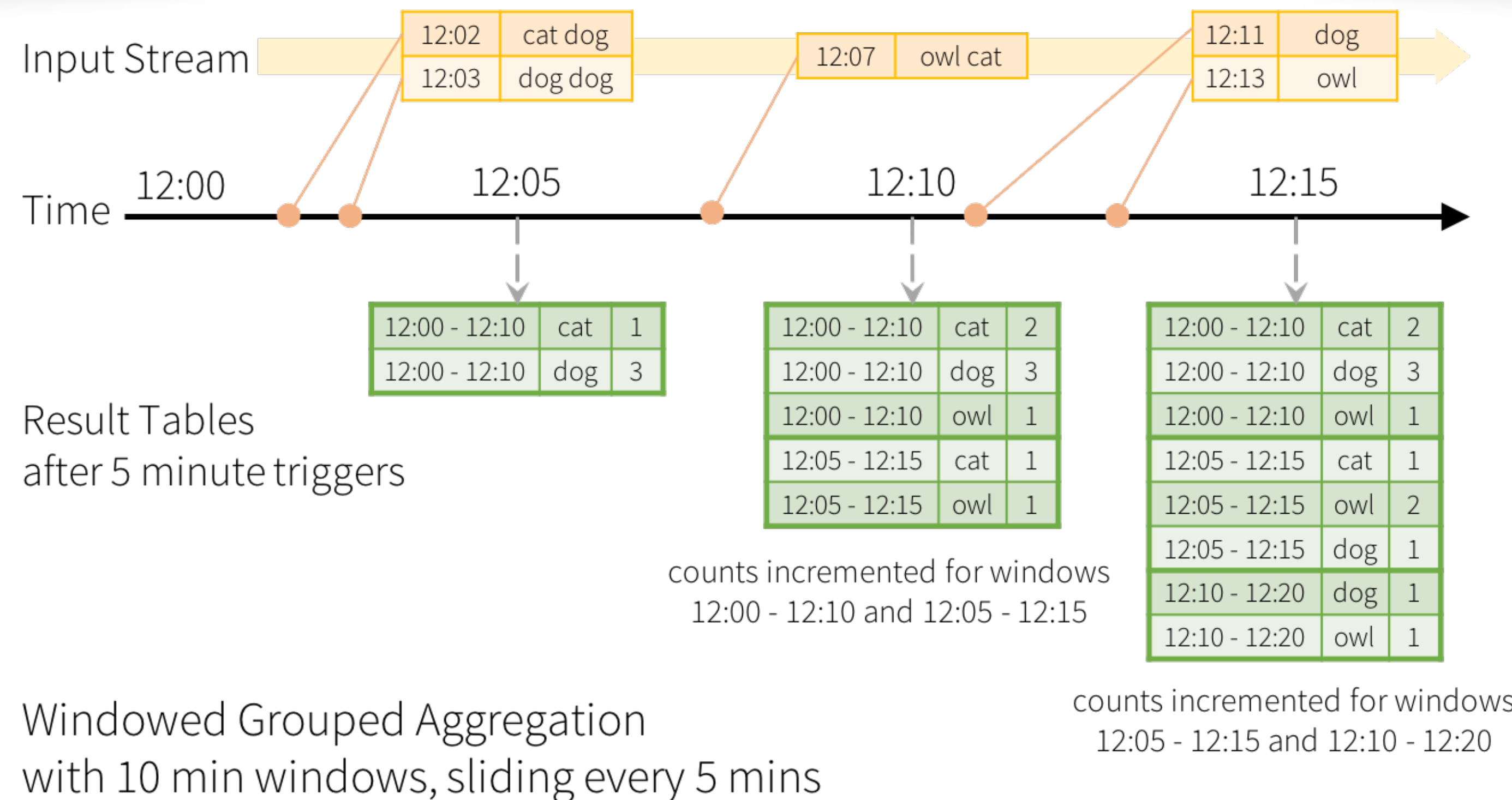


Spark Streaming (Structured Streaming)

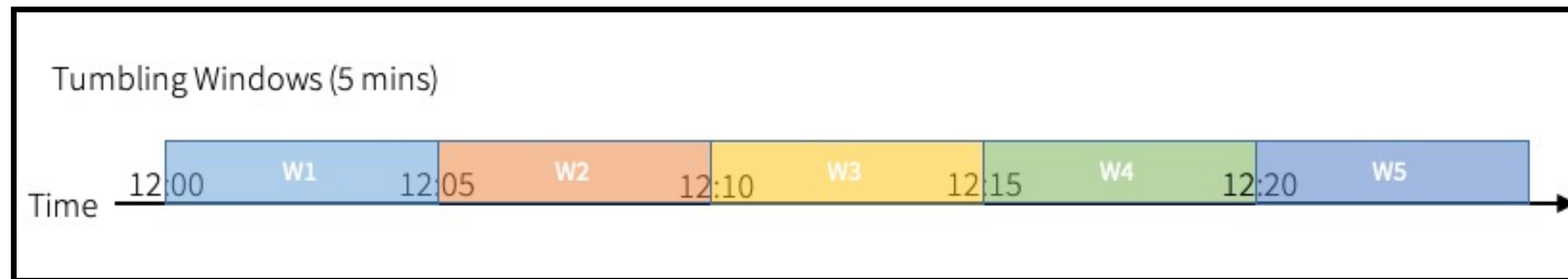
- ◎ **Structured Streaming does not materialize the entire table.**
 - ▶ It reads the **latest available data** from the streaming data source, **processes it incrementally** to update the result, and then **discards the source data**.
 - ▶ It only keeps the **minimum intermediate state data** required to update the result (e.g., intermediate counts in the earlier example).
 - ▶ As such, the user does not need to reason about fault-tolerance, and data consistency (at-least-once, or at-most-once, or exactly-once).
 - ▶ It delivers **end-to-end exactly-once** semantics under any failure by relying on checkpointing and write-ahead logs to record the offset range of the data being processed in each trigger.

Window Grouped Aggregation

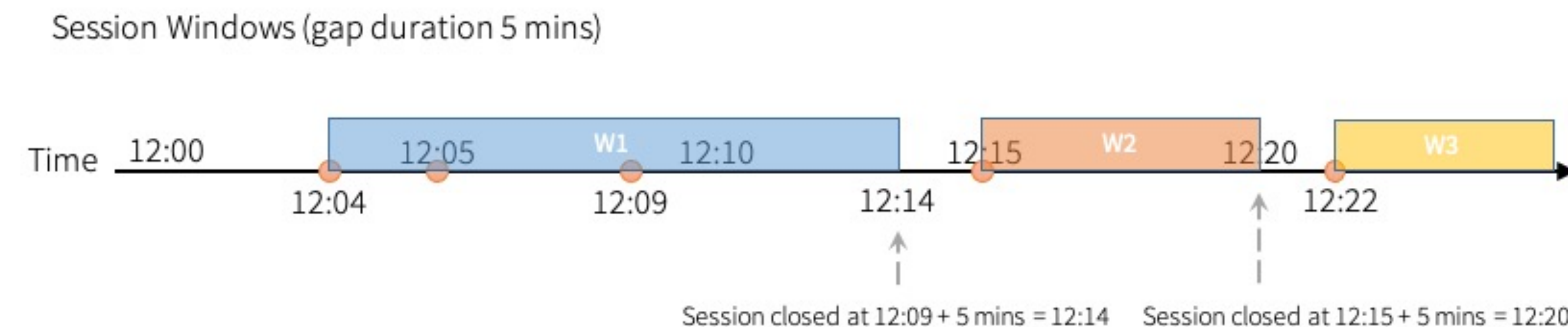
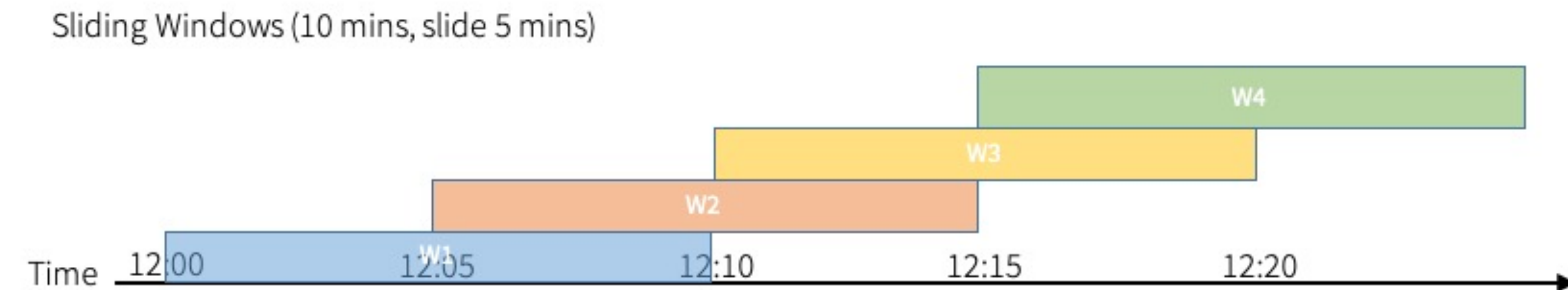
```
words = ... # streaming DataFrame of schema { timestamp: Timestamp, word: String }  
  
# Group the data by window and word and compute the count of each group  
  
windowedCounts = words.groupBy(window(words.timestamp, "10 minutes", "5 minutes"), words.word).count()
```



Types of time windows



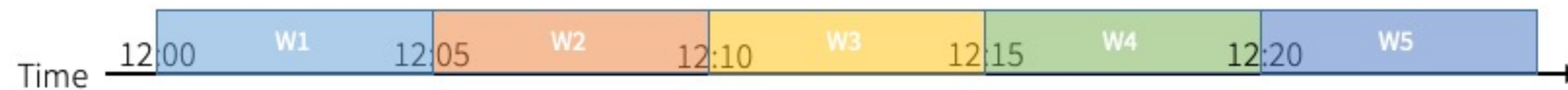
Tumbling windows are a series of fixed-sized, non-overlapping and contiguous time intervals. An input can only be bound to a single window.



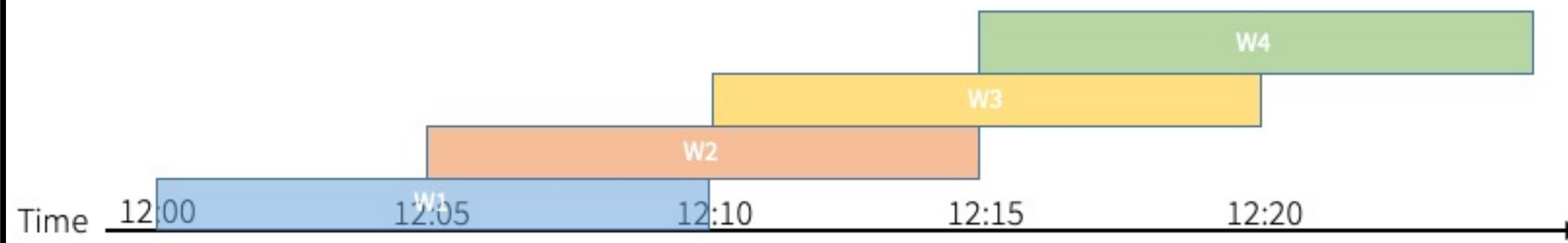
<https://spark.apache.org/docs/latest/structured-streaming-programming-guide.html#types-of-time-windows>

Types of time windows

Tumbling Windows (5 mins)



Sliding Windows (10 mins, slide 5 mins)



Sliding windows are similar to tumbling windows from the point of being “fixed-sized”, but windows can overlap if the duration of the slide is smaller than the duration of the window, and in this case, an input can be bound to the multiple windows.

Session Windows (gap duration 5 mins)



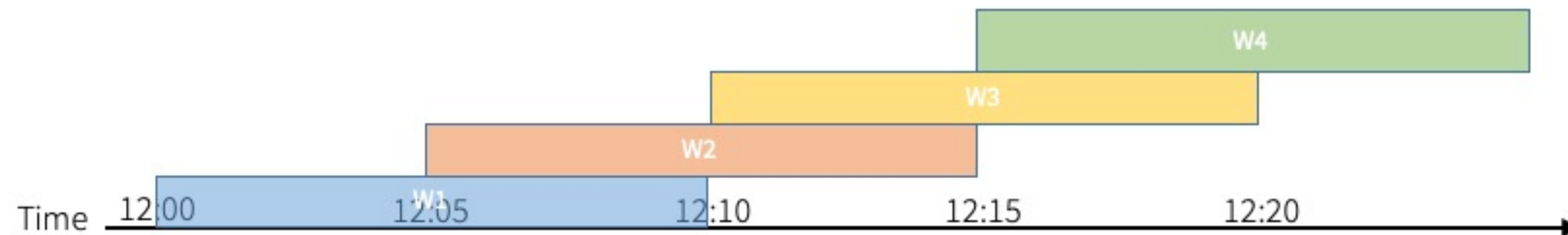
<https://spark.apache.org/docs/latest/structured-streaming-programming-guide.html#types-of-time-windows>

Types of time windows

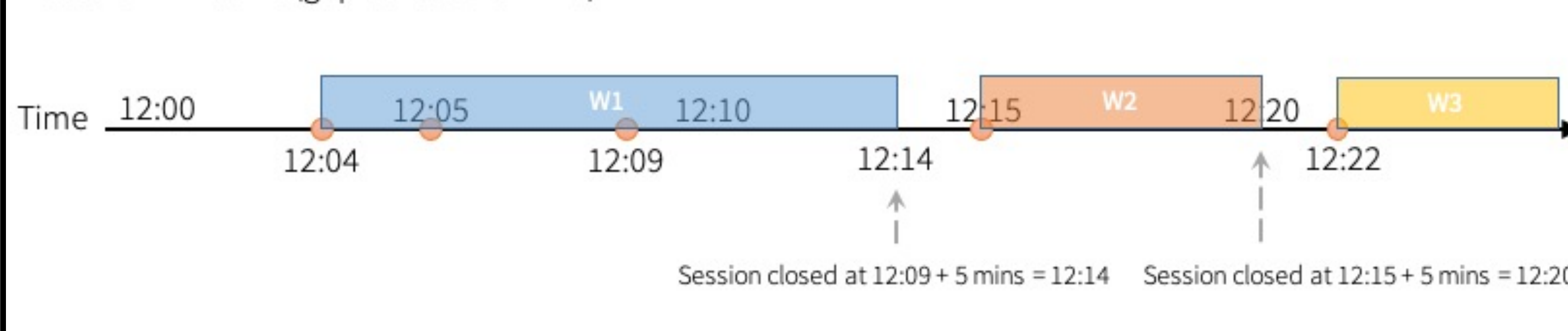
Tumbling Windows (5 mins)



Sliding Windows (10 mins, slide 5 mins)



Session Windows (gap duration 5 mins)



Session windows have a dynamic size of the window length, depending on the inputs. A session window starts with an input and expands if the following input has been received within the gap duration.

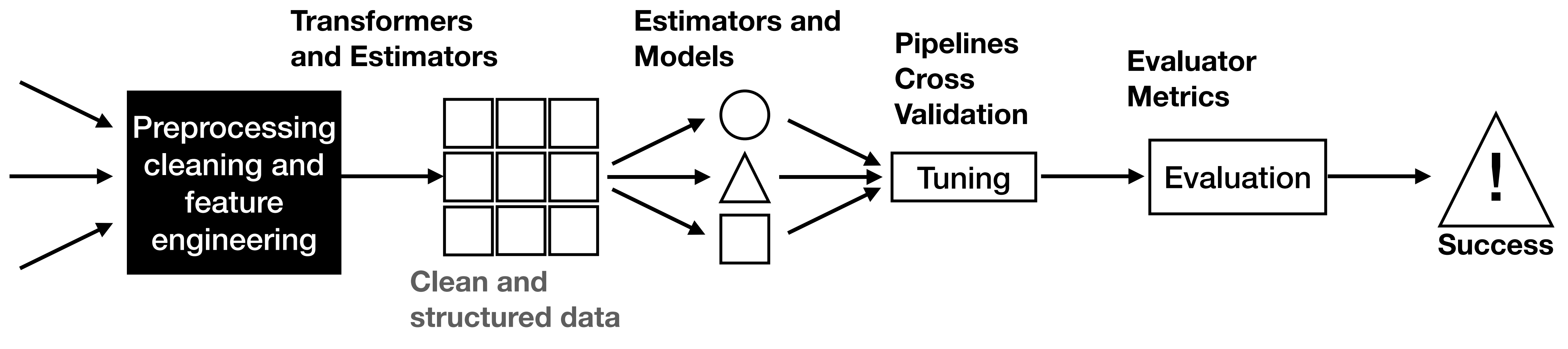
<https://spark.apache.org/docs/latest/structured-streaming-programming-guide.html#types-of-time-windows>

MLlib

- A tool for making practical machine learning scalable and easy. It provides tools such as:
 - **ML Algorithms**: common learning algorithms such as classification, regression, clustering, and collaborative filtering
 - **Featurization**: feature extraction, transformation, dimensionality reduction, and selection
 - **Pipelines**: tools for constructing, evaluating, and tuning ML Pipelines
 - **Persistence**: saving and load algorithms, models, and Pipelines
 - **Utilities**: linear algebra, statistics, data handling, etc.

Structured APIs

Raw Data



MLlib

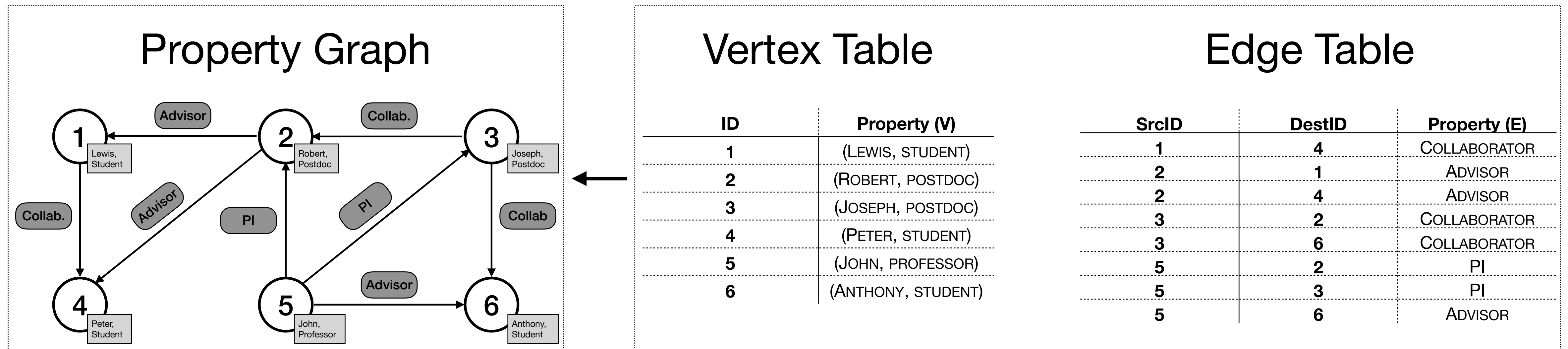
- ◎ **Scale vs. Single-Node:** Unlike scikit-learn (runs on a single machine), MLlib distributes data and computation across a cluster. This means MLlib can handle datasets or models too large for one machine's memory by scaling out horizontally.
- ◎ **Algorithm Availability:** MLlib provides a curated set of the most common algorithms optimized for distributed use covering the bases for big data (and is continually improving).
- ◎ **Performance Trade-offs:** On massive data (gigabytes to terabytes), solutions like scikit-learn will either crash or become unbearably slow, whereas MLlib can still train models by splitting work over the cluster.

GraphX

- ◎ GraphX unifies **ETL, exploratory analysis**, and **iterative graph computation** within a single system.
 - It helps view data as graphs and collections.
 - Allows for efficiently transforming and joining graphs with RDDs and writing custom iterative graph algorithms using the Pregel API.
 - It is fast, flexible and retains all the fault-tolerance and ease of use provided by Apache Spark.
 - Large collection of algorithms to choose from.
- ◎ GraphX represents data as a property graph where both **vertices** and **edges** can have **associated attributes (properties)**.

GraphX

- GraphX unifies **ETL**, **exploratory analysis**, and **iterative graph computation** within a single system.
 - It helps view data as graphs and collections.
 - Allows efficiently transforming and joining graphs with RDDs and writing custom iterative graph algorithms using the Pregel API.
 - It is fast, flexible and retains all the fault-tolerance and ease of use provided by Apache Spark.
 - It has a large collection of algorithms to choose from.
- GraphX represents data as a property graph where **vertices** and **edges** can have **associated attributes (properties)**.



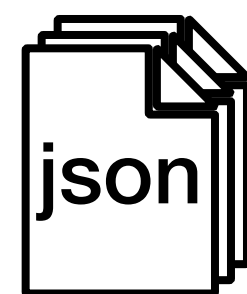
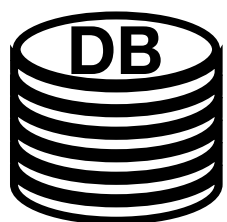
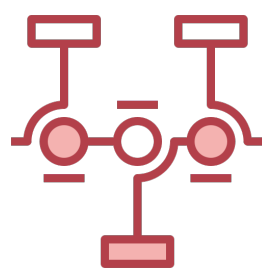
GraphX

- Integrates seamlessly with Spark SQL, MLlib, and other components, allowing for a more unified big data analysis pipeline.
- Includes an API inspired by Google's Pregel model that makes it easier to **write iterative graph algorithms** (like PageRank or connected components) by abstracting message passing between vertices.
- Implements optimizations such as **graph partitioning** (using techniques like **vertex cuts**) to balance the workload across the cluster.
 - Note that: none of the GraphX graph builders repartitions the graph's edges by default; instead, edges are left in their default partitions (such as their original blocks in HDFS).

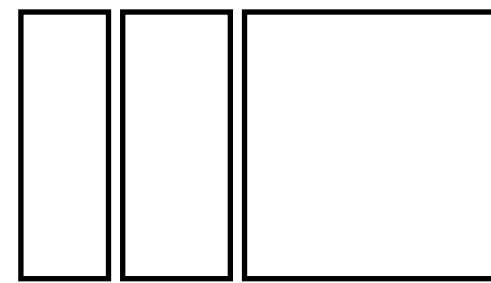
End-to-end Pipeline

Example 1

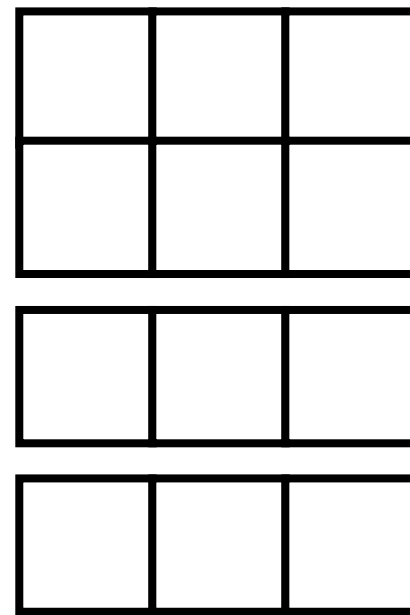
Input Sources



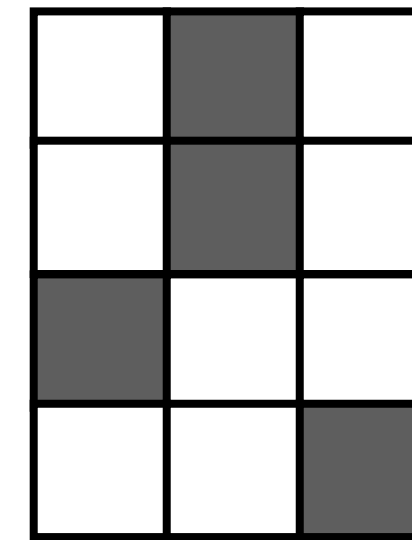
Data streaming



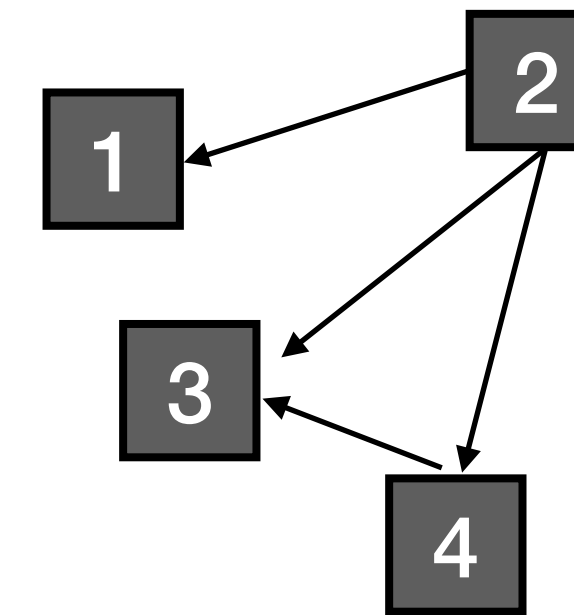
Structured Streaming



SQL Queries

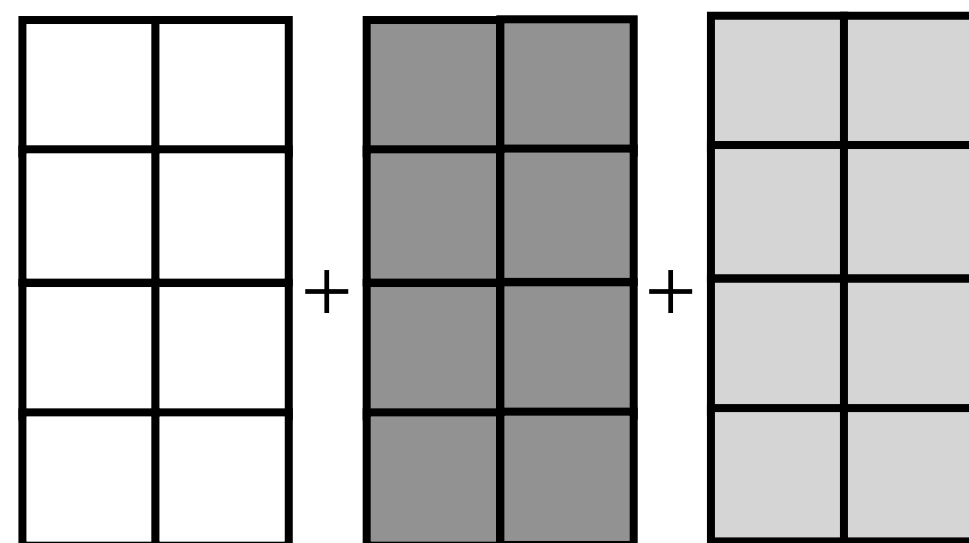


GraphX

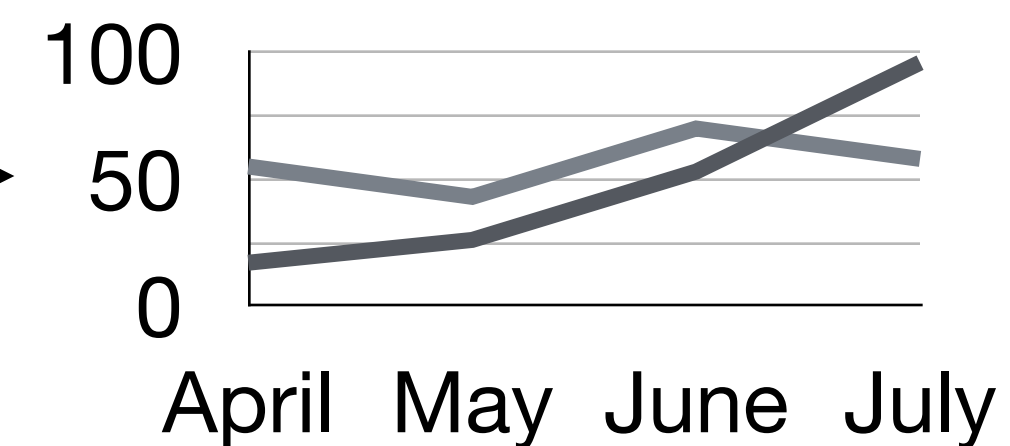
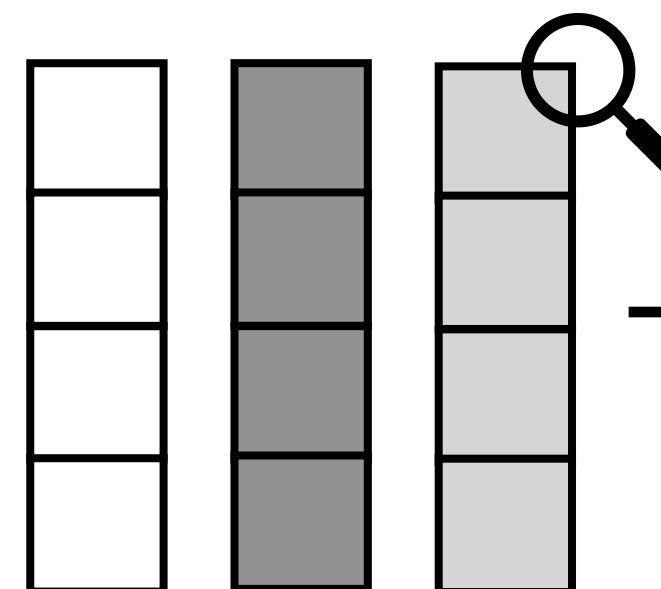


Example 2

SQL Queries



MLlib



More information

- Structured Streaming:

- ▶ Bill Chambers & Matei Zaharia - Spark: The Definitive Guide (Chapter 20 to 23)
- ▶ <https://spark.apache.org/docs/latest/structured-streaming-programming-guide.html>

- MLlib:

- ▶ Bill Chambers & Matei Zaharia - Spark: The Definitive Guide (Chapter 24 to 29)
- ▶ <https://spark.apache.org/docs/latest/ml-guide.html>

- GraphX:

- ▶ Bill Chambers & Matei Zaharia - Spark: The Definitive Guide (Chapter 30)
- ▶ <https://spark.apache.org/docs/latest/graphx-programming-guide.html>

Questions?